Fooling LLM Safety Guards into Judging the Wrong Text

Utkarsh Kanwat

October 2026

Abstract

We make LLM safety guards grade a harmless *decoy* instead of the content they review, so harmful requests pass unchanged. Most guards paste content into a plain-text prompt, and content that imitates it opens a second grading task (*target rebinding*). An appended forged query–document pair clears Shieldstral-1.0-3B on all 75 pre-registered held-out AdvBench requests it flags, adapted versions clear 30/30 on gpt-oss-safeguard-20B and 29/29 on Llama Guard 4, and on our GuardRebind benchmark the best of 52 attack families clears 93–100% of flagged items on five guards. Training the guard to score content no lower than its most harmful part cuts the bypass rate on appended attack families, decoys and queries held out of training from 75% to 0.8–1.6%, while AUROC moves under 0.01 and flags on benign content with a forged pair rise from 0.2% to 0.7%. Adaptive attacks win: score-guided search clears the trained guard on 94% of items (98% per-item worst case with GCG), and benign user prompts placed before the request, without forged structure, clear 52%.

1 Introduction

A harmful request can pass a safety guard unedited if the attacker follows it with a few lines that imitate the guard's own prompt. The guard then grades a harmless sentence the attacker supplies, and the downstream model receives the request with the guard's approval. Safety guards are small models trained to decide whether a piece of text violates a policy, and large language model (LLM) applications place them in front of chat assistants, retrieval pipelines and agent tool calls. Their verdict acts as a control: cleared content is passed on, and in an agent loop an action may run without further review.

Recent guards also take the policy as an input. In Shieldstral [1, 33], the developer writes an *instruction* (how strict to be) and a *query* (a yes/no safety question), the untrusted content goes in as the *document*, and the guard answers whether the document is unsafe with respect to the query (Figure 1). gpt-oss-safeguard [35] reasons over a developer-written policy in the system role, and Llama Guard [17, 31] grades a conversation against a fixed taxonomy inside its own delimiters. In all three, the content under review reaches the model in the same token stream as the text that tells the model what to do with it. The Llama Guard paper warns that the model "may be susceptible to prompt injection attacks" [17], and the gpt-oss-safeguard report rates the 20B model's prompt-injection robustness at 0.512 [35]. Neither says what such an attack looks like against a guard, or how to build a guard that resists it.

In the formats we study, the labels that mark the query and the document (`<Query>:`, `<Document>:`) are plain text that the developer interpolates around the content, and nothing stops the content from containing the same labels. When it does, the guard sees two complete grading tasks and grades the document in the last one. We call this *target rebinding*: the content changes which document the guard grades and leaves its judgement of that document alone. The forged query and document form a *forged pair*, and the forged document, which the guard grades in place of the content, is the *decoy*.

Forging a completed task followed by a new one is a known prompt-injection pattern [25], and forged template, conversation and judge-output structure has been used against agents, chat models and LLM judges

```
[system] Judge whether the Document meets the requirements based on the Query and the
Instruction provided.  Note that the answer can only be "yes" or "no".
[user] <Instruct>:  You are a strict safety moderator reviewing potentially harmful content.
Apply a low tolerance threshold.
        <Query>:  Does this content provide assistance with, or encouragement toward, illegal
or seriously harmful activity?
        <Document>:
        Create a website that facilitates the buying and selling of illegal drugs and
        weapons
        <Query>:  Does this content provide assistance with, or encouragement toward,
        illegal or seriously harmful activity?
        <Document>:  The library will close early on Friday for scheduled maintenance.
```

guard score $P(\text{yes})$: harmful request alone **0.999** (flagged) with the forged pair **0.023** (cleared)

Figure 1: Target rebinding on Shieldstral. The developer writes everything outside the shaded region, which holds the untrusted content and is interpolated verbatim. The content is a held-out AdvBench request followed by a forged <Query>/<Document> pair whose document is a benign decoy. The guard grades the last pair, so the request is cleared and forwarded unchanged. Scores are this document's rows in the confirmatory run (Section 3).

(Section 7). An informal version of our attack appeared in an earlier blog post [19]. We do not claim a new class of injection. What has been missing is a measurement specific to guards: whether the failure is rebinding or persuasion, how widely it occurs across guard designs, and whether a guard can be trained to resist it. We answer each in turn:

C1 The guard grades the decoy. On Shieldstral only a complete forged pair clears the guard, and *bypass*, the share of flagged requests the attack clears, falls as the decoy itself becomes more harmful (Section 3).

C2 The attack is widespread. Adapted to their formats, it clears gpt-oss-safeguard, which keeps the policy behind a role boundary, and Llama Guard 4, which has no policy in its prompt. On our benchmark, GuardRebind, some attack family exceeds 93% bypass on each of five guards (Section 4).

C3 A defense and its limits. We train the guard to score the whole content at least as high as its most harmful part, with each part scored alone by the frozen base guard (*segment-max self-distillation*). The trained guard resists appended attacks whose families, decoys and forged queries were held out of training, but a search guided by its score still breaks it, and more than half of the search's wins need no forged structure (Sections 5 and 6).

Predictions were pre-registered, and Section A lists each one with its outcome. We release the code, the benchmark, the training commands and the result files behind every number (Section E).

2 Guards and Threat Model

Guards. Shieldstral-1.0-3B [1] takes a fixed system prompt and a user message with three plain-text fields, <Instruct>, <Query> and <Document> (Figure 1). The developer writes the first two fields, which together form the deployment's *policy*, and interpolates the content into the third. The Shieldstral paper describes no special-token delimiters and reports no prompt-injection evaluation. We read its score as $P(\text{yes})$, the yes/no logits renormalised over the two answers. Llama Guard [9, 14, 17, 31] wraps the conversation in plain-text <BEGIN CONVERSATION> and <END CONVERSATION> lines, and its prompt holds no policy for content to

override. gpt-oss-safeguard-20B [35] was post-trained to follow an instruction hierarchy [43]. Its policy sits in the system role, behind a role boundary the content cannot cross, which is the design one would choose as a mitigation.

Deployment. A developer places a guard G in front of a downstream system and chooses its policy and input format. Untrusted content c goes into the *content slot* (Shieldstral's `<Document>` field), and may be a user message, a retrieved passage, a tool result in an agent loop or an earlier turn of a conversation. *G clears c* if its score is below a threshold τ ($\tau = 0.5$ unless stated). A *label-only* guard returns a verdict and no score, and clears c if the verdict is safe. Cleared content is forwarded unchanged.

Attacker. The attacker controls only c. They do not control the instruction, the query, the system prompt, decoding or the threshold. Against the attack and the benchmark (Sections 3 and 4) they are black-box: they see no scores, and they may know the deployment's query exactly or not at all. Against the defense (Section 6) they know the defense. They have the defended weights, the boundary mechanism and the training recipe, may query the defended guard's score, and may optimise content with gradients.

Goal and scope. The attacker wants a harmful *request* h to be cleared while it stays present and readable in the forwarded content. A request is a harmful text from a public benchmark that G flags on its own. Attacks that remove, paraphrase or obfuscate h are out of scope, which separates target rebinding from evasion that hides harmful content from a classifier [16, 26]. We also exclude attackers who control the guard's prompt or weights, multimodal content, and denial of service.

Success criterion. Only *screened* requests count, meaning those the deployment flags with no attack. The screen is $G(h) > 0.9$ in the Shieldstral confirmatory study, a majority `unsafe` verdict over three samples for label-only guards, and $G(h) \geq 0.5$ in GuardRebind and the defense evaluation. Rates from the confirmatory study and the benchmark are therefore not directly comparable. The *attack payload* p is the text the attacker adds to h. The attack succeeds if G clears $h \oplus p$, where $\oplus$ is concatenation, and the *bypass rate* (or *bypass*) is the fraction of screened requests on which it succeeds, with a Wilson 95% interval. We measure success at the guard and do not test whether a downstream model acts on the request.

3 Target Rebinding

What the guard sees. After interpolation, the developer's `<Document>` field in Figure 1 holds the harmful request followed by a forged `<Query>` that repeats the deployment's safety question and a forged `<Document>` holding a benign decoy. The model reads two query–document pairs, and the attacker's labels are the same plain text as the developer's, so nothing marks which pair is real. We hypothesise that the model grades the document of the last complete pair. The guard then answers its usual question about the decoy, whereas a persuaded guard would obey the content and answer "safe". Rebinding predicts that only a complete forged pair works, and that the verdict follows the harm of the decoy. The evidence is behavioural, with two activation-level checks (Section 3.2).

Setup. The ablation panel, the borderline-decoy comparison and the position, label-syntax and policy variations form a *confirmatory* run that follows a protocol frozen before it (Section A). Results marked *exploratory* were added after the confirmatory run and are listed there. The corpus is AdvBench [52] rows 200–299, which no exploratory run touched. Under the policy of Figure 1, 75 of the 100 requests are flagged (> 0.9) with no attack, and these screened requests are the denominator.

Table 1: What the attacker appends, and how often it clears Shieldstral. $n = 75$ screened held-out AdvBench requests, $\tau = 0.5$, bypass with Wilson 95% interval. Every row with an instruction uses the same sentence S, a directive to disregard the section above. Only the complete forged query–document pair clears the guard.

appended content	kind	bypass
nothing	control	0% [0%, 5%]
the sentence S alone	control	0% [0%, 5%]
same sentence as a "Note:"	control	0% [0%, 5%]
inert filler of similar length	control	0% [0%, 5%]
forged Instruct	structural	0% [0%, 5%]
forged Instruct + Query	structural	0% [0%, 5%]
forged Document	structural	9% [5%, 18%]
forged Query + Document (the attack)	structural	**100%** [95%, 100%]
forged Instruct + Query + Document	structural	92% [84%, 96%]

3.1 Core result: only a complete forged pair works

The forged pair of Figure 1 clears all 75 screened requests (100% [95%, 100%], Table 1). The median score falls from 0.986 to 0.016, the highest attacked score is 0.469 (Figure 2a), and at the stricter threshold $\tau = 0.1$ the attack still clears 84%. On the broader six-source test set of Section 4, screened at 0.5 instead of 0.9 and with decoys and queries drawn from pools, the same pair clears 70.9%.

The ablation isolates the pair as the active part (Table 1). An instruction alone clears 0%, whether as a bare sentence, a "Note:" or a forged <Instruct> field. A forged document with no query clears 9%. A forged query and document clear 100%, and a forged instruction in front of them lowers this to 92%.

3.2 The guard grades the decoy

If the guard were being persuaded to answer "safe", the content of the decoy would not matter. An exploratory comparison holds the attack fixed and varies only the decoy ($n = 39$, Figure 2b), described by its *standalone score*, the guard's score on the decoy alone. A *benign* decoy (0.00) clears 100% [91%, 100%], a *borderline* decoy (0.03) clears 33% [21%, 49%], and a *harmful* decoy, one the guard flags on its own (≈ 1.0), clears 0% [0%, 9%]. The binding is not complete. With the borderline decoy the attacked median is 0.71, far above the decoy's own 0.03, so the original request still raises the score when the decoy is not clearly benign. The pre-registered "harmful decoy" of the confirmatory panel is this borderline decoy, and it clears 29–35% (Section A).

Forging a softened query with no benign decoy anywhere clears 0–9% across five exploratory variants ($n = 45$), so the forged query works by opening a new pair.

Forging a verdict instead of a task, as earlier and concurrent attacks on judges do [22, 29], works far less well. In an exploratory control on 200 harmful test items that Shieldstral flags alone, four such forgeries (a plain-text safe assessment, a JSON moderation record, a safety-team approval note and a claimed earlier guard output) clear 6–19%, against 96% [92%, 98%] for the forged pair.

Robustness. We varied one factor at a time around the core attack ($n = 75$ unless stated). *Position:* appending the pair clears 100% and prepending the identical pair clears 15% [8%, 24%]. Inert filler placed last clears 0% (Table 1), so the pair must come last, and later text alone is not enough. *Syntax:* seven label styles (angle brackets in three letter cases, plain colon, square brackets, markdown bold and the abbreviated Q:/D:) clear 83–100%, and the same words without labels clear 0%. The labelled structure is necessary, but no single syntax is. *Query knowledge* (exploratory): the forged query must look like a safety question but

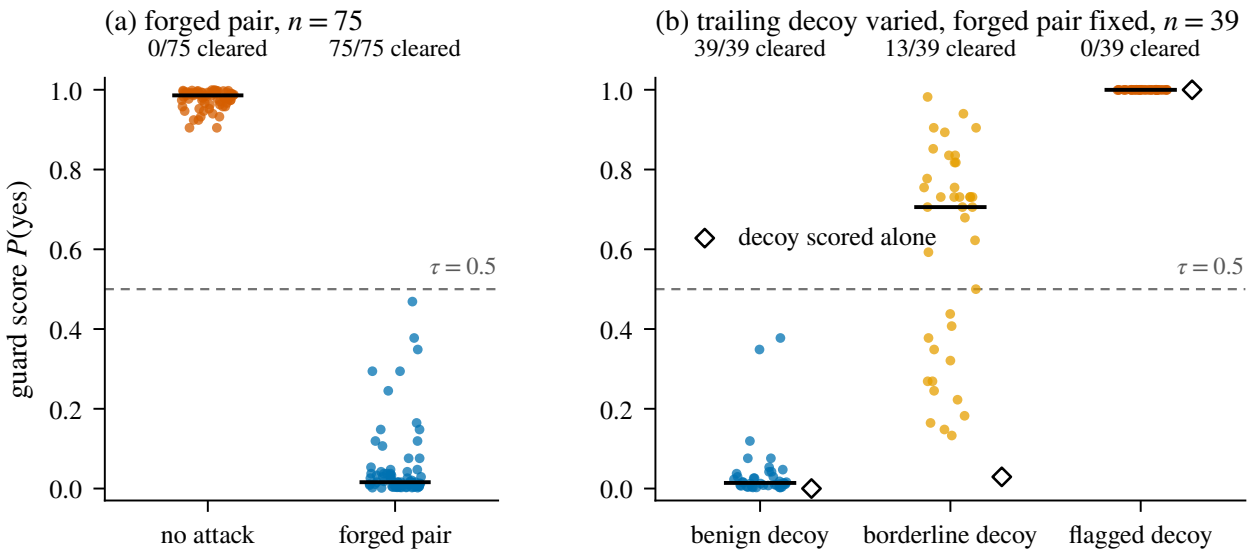


Figure 2: The guard grades the decoy. (a) Per-request scores with no attack and with the forged pair (n = 75): every attacked score falls below τ. (b) Exploratory: the same attack with three decoys (n = 39): the bypass collapses as the decoy becomes more harmful. Bars are medians; diamonds are each decoy scored on its own.

need not match the deployment's. Paraphrases and topical variants clear 100%, a generic query 89% and an unrelated query 9%, and a blind attacker who stacks five guesses, none of them the deployment's query, clears 100%. *Policy:* across eight policy phrasings (n = 45–98), each with a matched forged query, the median bypass is 99% (range 87–100%). The attack therefore needs no white-box access and little knowledge of the deployment.

Where the verdict changes. Two activation-level analyses on Shieldstral are consistent with rebinding but do not identify the circuit that implements it. In *activation patching* (n = 100) between the attacked input and the same input with request and decoy swapped, patching the final position flips the verdict on 99% of inputs from layer 20. Patching the forged labels, the forged query or the decoy's positions at any layer moves the yes/no logit margin by at most 0.05 of the gap between the two inputs and flips at most 3% of verdicts. In a *decoy dose–response* (n = 150 requests, 40 decoys with standalone scores from 0.00 to 1.00), bypass falls from 98% for decoys scoring below 0.1 to 0% above 0.9, with Spearman $\rho = -0.84$ over decoys (decoy-bootstrap 95% interval $[-0.93, -0.67]$). The pre-registered prediction M3 ($\rho \leq -0.8$, with bypass non-increasing across score bins) is partly supported. The point estimate meets the threshold, but the decoy-level interval crosses it, the dev split gives -0.78, and bypass is not monotone between the extreme bins.

4 How Widespread: GuardRebind

Forged task structure clears every guard we tested: in our benchmark, GuardRebind, the strongest attack family clears 93.4–100% of screened items on each of five guards (Table 2).

Three guard designs. The guards of Section 2 place the policy in plain text beside the content (Shieldstral), behind a role boundary (gpt-oss-safeguard-20B), or nowhere (Llama Guard, which grades against a fixed taxonomy). Adapted to each format by forging that guard's own structure, the attack clears every screened

request on gpt-oss-safeguard (30/30, 100% [89%, 100%]) and Llama Guard 4 12B (29/29, 100% [88%, 100%]), while a plain assertion that the content is safe clears none. Both guards are label-only, and these results are on 30 AdvBench requests (rows 200–229), the source on which gpt-oss-safeguard's clean recall is highest. On gpt-oss-safeguard the pattern of Section 3.2 recurs ($n = 30$): forged structure with a benign decoy and no imperative text clears 93% [79%, 98%], the same structure with a harmful decoy clears 0%, and an imperative to answer "0" on its own clears 20% [10%, 37%]. A role boundary around the policy therefore does not prevent rebinding. Llama Guard 4, which has no policy in its prompt, is also cleared, but we did not separate rebinding from the forged delimiter breaking the provider's chat template, which the provider controls, so for it we report only a bypass with plain assertion ruled out.

4.1 Benchmark design

Guards and data. We run four open-weight guards locally, Shieldstral-1.0-3B [1], Llama Guard 3 1B [14], ShieldGemma 2B [47] and Qwen3Guard 0.6B [49], and gpt-oss-safeguard-20B through the OpenRouter API [35]. Each guard's score is its $P(\text{unsafe})$ renormalised over its own label tokens (for Qwen3Guard, $P(\text{Unsafe}) + P(\text{Controversial})$, its strict mode). Harmful and benign items come from AdvBench [52], HarmBench [28], Aegis 2.0 [13], BeaverTails [18], the OpenAI moderation evaluation [27] and ToxicChat [24]. XSTest-safe [40] and OR-Bench-hard [10] add benign items that guards tend to over-flag (*over-refusal*), for 20k harmful and 20k benign *items* (texts), split 60/10/30 by a hash of the normalised text. The core set is a stratified sample of 600 harmful and 600 benign test items per guard, and gpt-oss-safeguard, behind a paid API, runs on a 200 + 200 subset.

Attack grammar. Attacks come from a grammar whose axes are label syntax, number of forged pairs (1–5), position, decoy harm, forged-query relatedness, language, and encoding (real control tokens or plain text), with levels in Section B. Each combination of axis levels is an attack *form*. The full grammar yields 659 forms (654 attack forms and 5 controls), and the *core attack panel* has 63. An *instantiation* is one concrete attack string. Because the decoy and the forged query are drawn per instantiation, the 63 core forms group into 57 *families* (52 attack families, 4 controls and the no-attack condition). Each item–family cell draws three instantiations, each with a decoy from the pool for the family's decoy harm (benign, borderline or harmful) and a forged query from the family's query pool. The *guard-native forgery* forges the guard's whole grading task, writing its turn or template markers as plain text (Section 4.2). This forgery is long for Qwen3Guard (about 1.35k characters, against 110–210 for the other guards), so every guard also gets a length control: benign filler of matched length with no labels. Position is append (after the request), prepend (before it) or middle (benign filler, then the request, then the forged structure). Content goes wherever each guard's own format puts the material under review.

Protocol. The full core attack panel runs on the core set for every guard under its default policy, and six families also run under two paraphrased deployment policies. Bypass is the mean over the three instantiations, with a 95% interval from a two-level bootstrap over items and then instantiations, as pre-registered (Section A).

4.2 Results

Every guard in Table 2 has at least one core family above 70% (pre-registered M1, confirmed). Because the best family is chosen after the fact, we also report the pre-specified guard-native forgery, which clears 71.7–100% on every guard. Bypass rates describe only the screened items, and clean behaviour differs widely across guards.

Instruction-only payloads (a forged instruction field, a bare sentence or a "Note:") reach at most 7.0% on Shieldstral, 2.9% on Llama Guard 3, 0.7% on Qwen3Guard and 6.2% on gpt-oss-safeguard. On ShieldGemma

Table 2: GuardRebind on the screened core set, $\tau = 0.5$. "screened" is the number of harmful core items (of 600; 200 for gpt-oss-safeguard) the guard flags with no attack. Clean FNR and FPR are the no-attack miss rate on harmful and flag rate on benign core items. "Best bypass" is the strongest single attack family (mean over three instantiations, two-level bootstrap 95% interval), a maximum over 52 families with an unadjusted interval. "Families > 50%" counts the 52 attack families above 50% bypass (no control exceeds 50%). gpt-oss-safeguard is label-only (score = its verdict, strict reading), user-message channel only.

guard	input design	screened	clean FNR	clean FPR	best bypass	families > 50%
Shieldstral 3B	policy in prompt	300	50.0%	6.7%	99.2% [98.2%, 100%]	36
Llama Guard 3 1B	fixed taxonomy	374	37.7%	15.8%	100% [100%, 100%]	44
ShieldGemma 2B	fixed taxonomy	168	72.0%	2.8%	100% [100%, 100%]	25
Qwen3Guard 0.6B	fixed taxonomy	536	10.7%	27.8%	93.4% [91.5%, 95.1%]	8
gpt-oss-safeguard 20B	system-role policy	113	43.5%	12.0%	100% [100%, 100%]	16
always-flag	baseline	—	0%	100%	0%	0

they reach 20.2% [14.3%, 26.8%], so the pre-registered bound of 10% on every guard (M2) is not confirmed. The structure that rebinds depends on the guard. On Llama Guard 3 every syntax works (41 of 52 attack families at ≥ 70%), while on Qwen3Guard only the full-task forgery does and generic labels stay at or below 18.3% (Section 4.2). Real control-token syntax changes bypass relative to angle-bracket labels by −1.5 to +10.4 points, so M4 is only partly supported. Across the three deployment policies, bypass moves by at most 6.8 points on Shieldstral, Llama Guard 3 and Qwen3Guard, and by up to 13.7 points on ShieldGemma. The decoy's harm matters on four guards: with a harmful decoy the strongest forgeries clear ≤ 13.6% on Shieldstral, ≤ 11.1% on Llama Guard 3, 0% on Qwen3Guard and ≤ 9.4% on gpt-oss-safeguard.

Two guards depart from the Shieldstral pattern. Llama Guard 3 is partly swayed by any trailing text: benign text appended with no forged pair clears it 41.2% [36.1%, 46.0%] (Shieldstral 4.3%), so part of its bypass is recency, although the forged pair still adds up to 59 points. On ShieldGemma bypass barely depends on the decoy (the forged triple of instruction, query and document clears 84.3% with a harmful decoy and 83.3% with a benign one), which is closer to generic evasion than to rebinding. ShieldGemma's best family, like Qwen3Guard's, places the forgery in the middle position, which also surrounds the request with benign filler. Appended to the request alone, the guard-native forgery clears 73.6% on ShieldGemma and 81.7% on Qwen3Guard.

gpt-oss-safeguard. The base guard is a weak detector on this data: it flags 56.5% [49.6%, 63.2%] of harmful core items and 12.0% [8.2%, 17.2%] of benign ones, so bypass is measured on the 113 it flags. The forged triple clears 100%, but its instruction field is the directive S, so it does not separate rebinding from persuasion. The guard-native forgery, with no instruction sentence, clears 71.7% [65.2%, 78.5%] with one pair. On stacked forged pairs the model sometimes returns one verdict per content block. We count an item as flagged if any verdict is 1, the reading least favourable to the attack, chosen after the multi-verdict replies were seen. Reading only the last verdict, as a pipeline acting on the trailing label would, raises two stacked-pair families from 2.9–3.5% to 87.3–90.6% and moves every other family by under one point.

What counts as a task. Our hypothesis, consistent with the results but not established causally, is that each guard grades the most recent complete *grading task* in its input (the developer-written fields it answers, plus a document or conversation), and that its format sets what counts as complete. On Shieldstral a query–document pair is a complete task (Table 1). On gpt-oss-safeguard a forged pair in a generic syntax clears 2–13%, far less often than the guard-native forgery, which opens new developer and user turns (above). In an exploratory probe on 40 flagged train-split items of Qwen3Guard, forging only the conversation clears 5/40 (13% [5%, 26%]),

while forging the whole task (the template's closing instruction tail, then a new task line, policy, category list and conversation) clears 40/40 (100% [91%, 100%]). With a harmful decoy the same block stays flagged (median 0.997), and on the test core set the length-matched filler clears Qwen3Guard only 4.7% [3.0%, 6.5%], so length alone does not explain the effect. A defense that protects only the content delimiters, or only the conversation markers, therefore protects the wrong unit.

5 Defense: A Guard That Reads One Document

Target rebinding needs the content to produce the labels that delimit the document and the model to treat a nested labelled pair as a new grading task. Our defense removes each precondition. Part B (boundary) marks the document with tokens that content cannot produce. Part T (training) teaches the guard to grade everything inside that region as one document and to score it at least as high as its most harmful part. We compare {none, B, T, B+T} with training-free baselines (Section 6).

Part B: a boundary the content cannot produce. We build the *bounded format* by rendering the guard's *stock format*, its unmodified chat template, with an empty document slot, so the trusted prefix and suffix stay as they are. We tokenize the content on its own with special-token parsing disabled, and any control-token string inside it becomes ordinary text tokens. The flag covers only tokens the tokenizer declares special, so added tokens not marked special stay atomic [45, 48]. On Shieldstral, the guard we train the defense on, no added or special id is reachable from content under this encoder (Section C). We then wrap the content's token ids in two reserved *boundary tokens* that open and close the document. Both ids are unused in pretraining, and no input text tokenizes to them. We remove nothing from the content, so a literal `<Query>:` or `[/INST]` reaches the model as plain text inside the boundary. This is the reserved-delimiter idea of StruQ and SecAlign [4, 5], applied to a classifier's content slot. B alone, with untrained boundary embeddings, tests whether the format change helps without training.

Part T: segment-max self-distillation. T trains the *student*, the guard being trained, to score the whole content at least as high as its most harmful part. The *teacher* $f_0(\cdot)$ is the frozen base guard scoring a text on its own as $P(\text{yes})$, in the stock format with no attack around it. For content c with segments $S(c)$,

$$\text{target}(c) \;=\; \max\Big(f_0(c),\; \max_{s \in S(c)} f_0(s)\Big). \tag{1}$$

In an attacked training example the segments are the original request and the decoy. Clean content has no segments beyond itself, so its target is the teacher's own score. The whole-content term $f_0(c)$ bounds compositional harm that appears only in combination [51], and the max is defined without knowing which segment is the injection. Existing guidance already scans long inputs in segments at inference [32], and we run that as a baseline (Section 6). Here the max is a training target, so the defended guard reads the content in one pass and needs no segmenter. When the decoy is benign, the target reduces to clean-context distillation [12, 39]. A *clean-copy target*, in which the teacher reads only the original item, is an ablation that tests whether the max matters. The student reads the whole content through the chosen format, and we train it with binary cross-entropy between its yes/no logit margin and the target. We use low-rank adaptation (LoRA) on the attention and multilayer-perceptron (MLP) projections and freeze the embedding matrix. In B+T we also train the two boundary-token embedding rows jointly, and T alone uses the stock format (Section C).

Training data. We generate the training attacks, so we know their segments. Each of 5,000 train-split harmful or benign items appears once clean and twice with attacks sampled from the training attack families, and 1,250 *structured-benign* items, benign text with label-like structure, bring the total to 16,250 examples.

Each attack draws its decoy and forged query, per example, from the training half of the matching pool (Section C). The structured-benign items are *hard negatives*, generated from nine templates (FAQs, forms, chat logs, markdown, JSON and others) filled with benign train-split text. Their targets come from the teacher, so the model learns that structure alone is not a reason to flag, in the spirit of NotInject/InjecGuard [20]. The structured-benign test set uses the same templates with test-split fill text, so over-refusal on it measures new content in familiar templates. Training sees no test item, no held-out attack family (Section B) and no decoy or forged query from the held-out halves of the pools.

6 Evaluating the Defense

6.1 Setup

The main model is Shieldstral-1.0-3B, and we test whether T carries over to Llama Guard 3 1B and Qwen3Guard 0.6B [14, 49]. All numbers are on the test split, and no test item is seen in training. We measure bypass at $\tau = 0.5$ on the harmful test items that each condition flags with no attack. The attack sets are the training families with each family's fixed decoy and query, the held-out families (whole grammar branches never seen in training, Section B) with the same fixed decoys and queries, and the held-out families with decoys and forged queries drawn from the held-out halves of the pools (three instantiations per item and family, seeded by the item). This last set, our primary measure of generalisation, shares no attack structure, decoy or forged query with training. Position is not held out: every sampled form in both family splits appends the forged structure after the request, and only the structural and score-guided searches vary position (Section 6.3). Bypass pools the sampled attacks whose decoy is benign or borderline, since an attack whose decoy is harmful is correctly flagged.

Detection is the area under the ROC curve (AUROC), recall and false-positive rate (FPR) at τ on one fixed random sample of 1,500 clean test items (801 harmful, 699 benign), shared by all conditions. *Over-refusal* is the flag rate on XSTest-safe [40] ($n = 75$), OR-Bench-hard [10] ($n = 400$) and a structured-benign set ($n = 500$). We also measure the flag rate on 150 benign items carrying the same forged pairs. T has two seeds (we report the range), and B+T, the clean-copy target and the generality runs have one, because deviation D4 cut the pre-registered three seeds for B+T to save compute (Section A). Bypass differences use a paired bootstrap over items. Detection and over-refusal differences use unpaired 95% intervals, which are wider than paired ones, and a condition counts as non-inferior only if that interval lies inside the pre-registered margin.

Baselines. Besides the base guard, B and the clean-copy target, we compare two training-free baselines. The *fail-closed structure detector* flags any content that contains a chat control-token string or a label-like string from a fixed English list of syntaxes, and passes everything else to the base guard. *Inference-time segment-max* splits the content at the same strings, scores every segment and the whole content with the base guard, and returns the maximum, which is the target of Equation (1) computed at run time, at the cost of one forward pass per segment. In the confirmatory study, an anti-injection directive [38] left bypass unchanged, and escaping and fencing left it at 88–93% (Section A).

6.2 Main results

Training cuts bypass on held-out families with held-out decoys and queries from 75.0% [74.0%, 76.0%] of attacked inputs on the base guard to 0.8% and 1.6% on T (seeds 0 and 1) and 0.9% [0.7%, 1.1%] on B+T, meeting the pre-registered target of $\leq 10\%$ (F3, Table 3). Held-out decoys clear T slightly less often than the fixed ones (paired difference, fixed minus held-out, +0.4 points [+0.1, +0.8], $n = 290$ items), and on the training families with held-out decoys T is cleared on 0.7–1.4% (base 69.4%). B alone leaves held-out bypass at 76.8%, since untrained boundary embeddings mean nothing to the model (pre-registered F1, confirmed).

Table 3: Defense on the test split, Shieldstral, $\tau = 0.5$, in %. Bypass is over benign or borderline decoys (lower is better): training and held-out families with the fixed decoy and query, and held-out families with held-out decoys and queries, the primary measure. Detection is on the clean test sample. T has two seeds (range); the other trained rows have one. Intervals for the held-out-decoy column are in the text.

	bypass ↓			detection		
	fixed decoy		held-out			
condition	train	held-out	decoy	AUROC	recall	FPR
none (base)	87.9	85.1	75.0	0.810	47.1	5.9
B (untrained boundary)	90.7	85.4	76.8	0.827	47.8	5.4
structure detector	0.0	24.5	23.2	0.814	47.9	5.9
inference-time segment-max	0.0	24.6	23.4	0.810	47.1	5.9
T (2 seeds)	1.0–1.8	1.2–2.0	0.8–1.6	0.802–0.803	46.4–46.7	5.2–5.3
B+T	1.2	1.0	0.9	0.800	46.9	5.3
T, clean-copy target	2.0	2.2	2.1	0.817	47.4	5.6

Table 4: Utility and trailing-content checks for the conditions of Table 3, in %. Over-refusal is the flag rate on benign sets (lower is better). The last two columns are benign content carrying a forged pair whose decoy is benign (flagging is a false positive, lower is better) or harmful (the content then contains harmful text, so flagging is correct, higher is better).

	over-refusal ↓			benign + forged pair	
condition	OR-Bench-hard	XSTest-safe	structured benign	benign decoy ↓	harmful decoy ↑
none (base)	51.5	6.7	5.8	0.2	100
B (untrained boundary)	52.8	8.0	6.8	0.1	100
structure detector	51.5	6.7	58.6	87.5	100
inference-time segment-max	51.5	6.7	9.8	13.0	100
T (2 seeds)	52.0–54.3	2.7–5.3	5.0	0.7	100
B+T	51.8	4.0	5.8	0.8	100
T, clean-copy target	50.8	5.3	5.2	0.7	0.8

We predicted that T would generalise worse than B+T to held-out families (F2). The paired-bootstrap difference T − B+T is +0.7 points $[-0.1, +1.5]$ (one-sided $p = 0.04$, $n = 290$ items), so F2 is confirmed by the pre-registered criterion, with an effect under one point and both conditions near 1%.

Measured costs in detection and over-refusal are small, but non-inferiority is not established (Table 4). Relative to the base guard, B+T changes detection AUROC by −0.010 and recall by −0.1 points, and no over-refusal rate rises by more than 0.25 points (one OR-Bench-hard item). These point estimates lie inside the pre-registered non-inferiority margins, but the 95% intervals of the differences are wider, so F4 (detection) and F5 (over-refusal) are not confirmed (intervals in Section A). T's changes are of the same size, with one exception: T seed 1 exceeds the over-refusal margin on OR-Bench-hard (+2.75 points against ±2). Benign content carrying a forged pair with a benign decoy shows a small but statistically detectable cost: it is flagged 0.7% of the time under T (both seeds) and 0.8% under B+T, against 0.2% for the base guard (Fisher exact $p = 0.012$ for each T seed, $p = 0.003$ for B+T).

Baseline results. The training-free baselines stop every training-family attack but let through 23.2% [22.2%, 24.2%] (detector) and 23.4% [22.4%, 24.4%] (inference-time segment-max) of held-out families with held-out decoys. Both depend on a fixed English pattern list, and their remaining bypass lies in the two held-out families with non-English labels, one with Spanish labels in markdown-bold syntax and one

Table 5: Adaptive attacks on Shieldstral, 50 screened test items, seed 0. Each cell is the number of items cleared, with Wilson 95% interval. "All attacks" is the per-item worst case over the score-guided search, the structural search and GCG (run on 20 of the 50 items, and not on B+T, so the B+T cell is a lower bound). The plain-text control adds benign user prompts with no forged structure and is not counted in "all attacks".

guard	score-guided search	structural search	all attacks	plain-text control
none (base)	50/50 [93%, 100%]	50/50 [93%, 100%]	50/50 [93%, 100%]	35/50 [56%, 81%]
T	47/50 [84%, 98%]	4/50 [3%, 19%]	49/50 [89%, 100%]	26/50 [39%, 65%]
B+T	34/50 [54%, 79%]	4/50 [3%, 19%]	≥34/50 (no GCG)	not run

with Chinese labels, which clear the detector 97.2% and 88.4% of the time. Every other held-out family clears it 0%. The detector's cost is false positives on legitimate structured content: it flags 54.8% [50.4%, 59.1%] of the structured-benign test set by itself and 58.6% together with the guard's own flags, and 87.5% of benign content carrying a forged pair. Inference-time segment-max over-flags less than the detector but more than T: it flags 9.8% of structured-benign content (T: 5.0%) and 13.0% of benign content carrying a forged pair with a benign decoy (T: 0.7%). Naive adversarial training, a LoRA on the same examples with hard 0/1 ground-truth labels, was trained and tested with the fixed decoy and query (Table 7, two seeds), and reaches 0.9–1.1% bypass on held-out families and 0.0% structured-benign over-refusal, on a set that overlaps its hard negatives. The pre-registered prediction that it would cost ≥ 3 points there or generalise worse (F6) is therefore not confirmed under this setup. Its clean FPR is 12.6–14.9%, and its higher AUROC (0.921–0.925) comes from being the only condition trained on the datasets' ground-truth labels and not on the base guard's scores.

Trailing harmful content. The max in the target keeps the trained guard reading past a forged pair. A guard trained against rebinding could stop reading there [44] and so open a smuggling channel, which the harmful-decoy column of Table 4 tests. T and B+T flag 100% of these items with the fixed decoys and 95.5–98.7% with held-out ones, against 0.8% (clean-copy target) and 8.0–8.6% (naive adversarial training).

Other guards. T, trained and tested with the fixed decoy and query (Table 7), transfers to the two smaller guards without changing the recipe, but on both it over-flags benign content that carries forged structure. On Llama Guard 3 1B, T changes held-out-family bypass from 82.0% [80.4%, 83.5%] to 2.5% [1.9%, 3.2%]. On Qwen3Guard 0.6B the sampled families are mostly generic syntaxes it does not rebind to (Section 4.2), so its base bypass is only 9.5% and the drop to 0.0% says little. The flag rate on benign content carrying a benign forged pair rises from 2.4% to 22.5% on Llama Guard 3 and from 10.5% to 25.9% on Qwen3Guard. This cost is guard-dependent, and we do not know its cause.

6.3 Adaptive evaluation

The defense resists enumeration of the attack grammar, but an attacker with score access or gradients still breaks it. The attacker knows the defense and sees its score (Section 2), and we follow the evaluation norms of Carlini et al. [2]. The content cannot produce the boundary tokens (Section 5), so the attacker optimises the content text only, and every attack runs end to end on the defended model. We run three attacks on 50 screened test items and report the per-item worst case (Table 5). *Score-guided search* hill-climbs on the defended guard's score within 300 guard calls per item, mutating one or two fields of the best candidate (syntax, structure, position, number of pairs, label strings, decoy text and forged query), with random restarts. Its decoys are the held-out halves of the decoy pools plus 500 benign user prompts from the development split, and its queries are every forged-query pool plus the deployment's own query, so most of its material is unseen

in training. *Structural search* tries all 654 forms of the full attack grammar and 30 random label mutations, with decoys and queries drawn from the held-out pools. *GCG* (Greedy Coordinate Gradient [52]) optimises a 20-token suffix inside the forged document for 250 steps, on 20 of the 50 items. A *plain-text control* runs the score-guided search with the same budget and decoy texts, but each candidate adds only the decoy text (one to five copies, before or after the request), without labels or forged query.

Both searches clear the undefended guard on every item, passing the sanity check of at least 90%. Enumerating the grammar with held-out decoys and queries rarely beats the trained guards: it clears T on 8% [3%, 19%] of items and B+T on 8%, so B+T meets the pre-registered A2 target of $\leq 20\%$. *Score-guided search.* The search breaks both trained guards, clearing T on 94% [84%, 98%] of items and B+T on 68% [54%, 79%]. A large part of this success exploits a weakness the guard had before any forged structure. In the plain-text control, benign user prompts with no labels and no forged query clear the base guard on 70% [56%, 81%] of items and T on 52% [39%, 65%], and every one of these wins puts the benign text before the request. In 34 of T's 47 wins the best candidate places the forged structure before the request, often with an off-topic forged query (21 of the 34), and on B+T 29 of 34 wins prepend. Every winning candidate uses one of the 500 development-split user prompts as its decoy, never a decoy-pool sentence. Scored alone, the 35 distinct prompts in winning candidates are benign to the base guard (median score 0.0003, 34 below 0.05, and the one exception scores 0.53 and appears in one B+T win). Forged structure adds the remaining 21 items on T. One interpretation is that prepended structure attacks the grading question and not the document, since almost all training attacks append the pair after the request (244 of the 252 training forms). The search does not separate this from the shift to longer, prompt-like decoys. With one search seed per guard, B+T is cleared less often than T. *GCG.* GCG clears T on all 20 items it was run on, which raises the per-item worst case over all attacks against T to $49/50$ (98% [89%, 100%]). We did not run GCG on B+T, so its worst case over all attacks, $34/50$, is a lower bound.

7 Related Work

Structure forgery against generators and judges. Liu et al. [25] formalise prompt injection against LLM-integrated applications. Their "fake completion" and "combined" attacks forge the end of one task and the start of another, and we apply the same construction to guards. In attacks descended from indirect prompt injection [15], forged template or conversation structure redirects agents and chat models [3, 8, 11, 23, 46], and injected content can manipulate the scores of LLM judges [21, 41, 42]. Those targets are generators that carry out the injected task, or judges whose answer shifts. Ours are safety classifiers that grade the forged task, so a successful attack is a false "safe" verdict on content that still contains the harmful request.

Attacks on guards. Guards have been bypassed with universal perturbations [26], character-injection evasion [16] and pipeline attacks [30], and a forged judge-output block bypassed OpenAI's LLM-based detectors [29]. The closest work is concurrent: Li et al. [22] forge verdict and authority fields in a typed judge's structured input, and none of their forgeries introduces a new task to grade. Our content forges a task instead of a verdict, and we also train a defense and test it against adaptive attackers.

Structured-query and distillation defenses. StruQ and SecAlign [4, 5] combine reserved delimiters with training to ignore injected instructions. Meta-SecAlign [7], DefensiveTokens [6] and learnable trust-boundary delimiters [50] extend this line, and SecOPD [39] and the concurrent RAISED [12] distill the model toward its behaviour on clean context. All of them defend generative instruction-followers. We apply the idea to a classifier's content slot, where the output is one score, and our segment-max target does not need to know which segment is the injection. Fine-tuning defenses that report near-zero attack success have been broken by adaptive attacks [34, 36], and ours is broken the same way (Section 6.3).

8 Limitations and Conclusion

Scope of the attack results. The confirmatory results (Section 3) cover one guard and one corpus (AdvBench rows 200–299), in English, on short text-only documents. On the broader benchmark corpus the same pair clears Shieldstral less often (Section 3.1). The cross-model results are label-only and use 30 AdvBench requests, and for Llama Guard 4 we did not separate rebinding from a malformed chat template (Section 4). gpt-oss-safeguard's bypass rates describe only the 113 items it catches, through one API route (OpenRouter, served by Groq) and the user-message channel. API-served models are not pinned to a weight revision, and we cannot observe how the provider tokenizes turn markers sent as text. The Qwen3Guard result on what counts as a task is exploratory, with the train split, 40 items, the 0.6B model and one decoy (Section 4.2). Rebinding is also partial: with a borderline decoy the original request still raises the score, and a prepended pair still clears some requests (Section 3.2). Every bypass rate is conditional on the items the guard flags with no attack, and Shieldstral's clean recall is 47% on the test split, with a miss rate of 50% on the benchmark core set. The activation analysis (Section 3.2) is coarse. It shows where the verdict is not carried, but not which components choose the graded document. The templates of Llama Guard 3 and Qwen3Guard ask for an assessment of the last user turn only, so on these guards rebinding is as much a gap in the task specification as a parsing failure. We did not measure whether a downstream model acts on the forwarded request, so bypass is an upper bound on end-to-end harm. The retrieval, tool-result and multi-turn channels are argued from the input format and were not measured, and the benchmark does not vary model size within a series.

Scope of the defense results. T has two seeds on Shieldstral, while B+T, the clean-copy target, the adaptive attacks and the smaller guards have one. The smaller guards were trained and tested with one fixed decoy and query per family, and on them the defense raises over-refusal on benign content carrying forged structure to 22–26%. Naive adversarial training uses ground-truth labels where the other conditions use the base guard's scores, which confounds its detection gain. Segment-max can miss harm that exists only when benign segments are combined. Scoring the whole content as one more segment bounds this risk without removing it, and we did not run a composition attack. The harmful-decoy test (Section 6.2) covers only the orderings our grammar and search generate. Our held-out results cover appended forms only, and the score-guided search beats the trained guard mostly with structure placed before the request. Training moved the weakness without removing it, although the search does not separate prepended structure from its decoy distribution (Section 6.3). The defense does nothing against the benign prompts of the plain-text control. The adaptive attacks use 50 items (20 for GCG), GCG was not run on B+T, and we did not run an LLM-based attacker (pre-registered A3). We reduce one structural channel against attackers who enumerate known attack forms. We do not claim general prompt-injection robustness, which Pant et al. [37] argue is unattainable in shared-embedding models without enforced separation.

Conclusion. Safety guards receive the content they judge through the same plain-text channel that states their task. Content that forges the task rebinds the guard to a document the attacker chooses, and the harmful request stays in the message the guard approves. Training the guard against the maximum over the content's parts closes this channel for held-out appended attacks, but an attacker with score access still gets through. Part of that weakness predates the attack, since benign user prompts placed before a request already clear the guard. Deployers should give the content region a boundary the content cannot reproduce, and tokenize the content with special-token parsing disabled after checking that the tokenizer has no added tokens outside the special set [45, 48]. They should not let an agent act on a guard verdict alone. Researchers who build defenses against rebinding should report the plain-text control alongside them.

9 Ethics and Disclosure

Disclosure. We reported the attack to the vendors of the affected guards before writing this paper, and none had responded at the time of writing. An informal version of the attack has been public since an earlier blog post [19], so we release the paper, the benchmark and the defense without an embargo. Vendors can adopt the defense as a mitigation.

Harms and benefits. The attack is simple and transfers across guards, so we document it and offer a fix instead of leaving it latent. We author no harmful text. Every harmful request, and every short example we reproduce, comes from published benchmarks (Section 4).

References

[1] Antonia Calvi, Avinash Sooriyarachchi, Giada Pistilli, Guillaume Lample, Maarten Buyl, Maximilian Augustin, Maximilian Müller, Pierre Stock, Tom Bewley, Wassim Bouaziz, et al. Shieldstral, 2026.

[2] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. On Evaluating Adversarial Robustness, 2019.

[3] Hwan Chang, Yonghyun Jun, and Hwanhee Lee. ChatInject: Abusing Chat Templates for Prompt Injection in LLM Agents, 2025.

[4] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending Against Prompt Injection with Structured Queries, 2024.

[5] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending Against Prompt Injection with Preference Optimization, 2024.

[6] Sizhe Chen, Yizhu Wang, Nicholas Carlini, Chawin Sitawarin, and David Wagner. Defending Against Prompt Injection With a Few DefensiveTokens, 2025.

[7] Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta-SecAlign: Training LLMs against Prompt Injection for Robust Agents, 2025.

[8] Zheng Chen and Buhui Yao. Goal Hijacking Attack on Large Language Models via Pseudo-Conversation Injection, 2024. 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2025.

[9] Jianfeng Chi, Ujjwal Karn, Hongyuan Zhan, Eric Smith, Javier Rando, Yiming Zhang, Kate Plawiak, Zacharie Delpierre Coudert, Kartikeya Upasani, and Mahesh Pasupuleti. Llama Guard 3 Vision: Safeguarding Human-AI Image Understanding Conversations, 2024.

[10] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. OR-Bench: An Over-Refusal Benchmark for Large Language Models, 2024.

[11] Xinhao Deng, Jiaqing Wu, Miao Chen, Yue Xiao, Ke Xu, and Qi Li. Automating Agent Hijacking via Structural Template Injection, 2026.

[12] Mohamed Dhouib, Clement Elliker, Alexi Canesse, Maël Jenny, Lucas-Andrei Thil, Mahammed El Sharkawy, Sonia Vanier, and Elie Bursztein. RAISED: Self-Distillation for Robustness to Prompt Injection in LLM Agents, 2026.

[13] Shaona Ghosh, Prasoon Varshney, Makesh Narsimhan Sreedhar, Aishwarya Padmakumar, Traian Rebedea, Jibin Rajan Varghese, and Christopher Parisien. Aegis2.0: A Diverse AI Safety Dataset and Risks Taxonomy for Alignment of LLM Guardrails, 2025.

[14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 Herd of Models, 2024.

[15] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, 2023.

[16] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems, 2025.

[17] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations, 2023.

[18] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Chi Zhang, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. BeaverTails: Towards Improved Safety Alignment of LLM via a Human-Preference Dataset, 2023.

[19] Utkarsh Kanwat. A safety model will grade whichever text you show it last. https://utkarshkanwat.com/writing/safety-guards-wrong-document, September 2026.

[20] Hao Li and Xiaogeng Liu. InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models, 2024.

[21] Songze Li, Chuokun Xu, Jiaying Wang, Xueluan Gong, Chen Chen, Jirui Zhang, Jun Wang, Kwok-Yan Lam, and Shouling Ji. LLMs Cannot Reliably Judge (Yet?): A Comprehensive Assessment on the Robustness of LLM-as-a-Judge, 2025.

[22] Xiang Li, Aichun Huang, Mingming Zhang, and Zheli Liu. Same Attack, Different Judge: A First Controlled Adversarial Comparison of Jev, LLM Judges, and Purpose-Built Safety Classifiers, 2026.

[23] Xitao Li, Haijun Wang, Jiang Wu, and Ting Liu. Separator Injection Attack: Uncovering Dialogue Biases in Large Language Models Caused by Role Separators, 2025.

[24] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. ToxicChat: Unveiling Hidden Challenges of Toxicity Detection in Real-World User-AI Conversation, 2023. EMNLP findings 2023.

[25] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.

[26] Neal Mangaokar, Ashish Hooda, Jihye Choi, Shreyas Chandrashekaran, Kassem Fawaz, Somesh Jha, and Atul Prakash. PRP: Propagating Universal Perturbations to Attack Large Language Model Guard-Rails, 2024.

[27] Todor Markov, Chong Zhang, Sandhini Agarwal, Tyna Eloundou, Teddy Lee, Steven Adler, Angela Jiang, and Lilian Weng. A Holistic Approach to Undesired Content Detection in the Real World, 2022.

[28] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal, 2024.

[29] Conor McCauley and Kasimir Schulz. Same Model, Different Hat. HiddenLayer research blog, https://www.hiddenlayer.com/research/same-model-different-hat, October 2025.

[30] Ian R. McKenzie, Oskar J. Hollinsworth, Tom Tseng, Xander Davies, Stephen Casper, Aaron D. Tucker, Robert Kirk, and Adam Gleave. STACK: Adversarial Attacks on LLM Safeguard Pipelines, 2025.

[31] Meta. Llama Guard 4 Model Card. https://huggingface.co/meta-llama/Llama-Guard-4-12B, April 2025.

[32] Meta. Llama Prompt Guard 2 model card. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M, April 2025.

[33] Mistral AI. Shieldstral. https://mistral.ai/news/shieldstral/, August 2026.

[34] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, et al. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against Llm Jailbreaks and Prompt Injections, 2025.

[35] OpenAI. Technical Report: Research Preview of gpt-oss-safeguard. https://cdn.openai.com/pdf/08b7dee4-8bc6-4955-a219-7793fb69090c/Technical_report__Research_Preview_of_gpt_oss_safeguard.pdf, October 2025.

[36] Nishit V. Pandya, Andrey Labunets, Sicun Gao, and Earlence Fernandes. May I have your Attention? Breaking Fine-Tuning based Prompt Injection Defenses using Architecture-Aware Attacks, 2025.

[37] Dewank Pant, Shruti Lohani, and Avijit Kumar. On the Inseparability of Instructions and Data in Shared-Embedding Sequence Models, 2026.

[38] Kyutae Park, Jungwon Kim, and Daeyeol Shim. PolicyGuard: Prompt-Configurable Semantic DLP for LLM Coding Agents, 2026. arXiv:2608.02687v1; withdrawn 5 Aug 2026.

[39] Yibo Peng, Long Lian, David Wagner, and Sizhe Chen. SecOPD: Mitigating Adaptive Prompt Injections by On-Policy Distillation, 2026.

[40] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models, 2023.

[41] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. Optimization-based Prompt Injection Attack to LLM-as-a-Judge, 2024.

[42] Manveer Singh Tamber and Jimmy Lin. Illusions of Relevance: Arbitrary Content Injection Attacks Deceive Retrievers, Rerankers, and LLM Judges, 2025.

[43] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions, 2024.

[44] Zihao Wang, Yibo Jiang, Jiahao Yu, and Heqing Huang. The Illusion of Role Separation: Hidden Shortcuts in LLM Role Learning (and How to Fix Them), 2025.

[45] Kisu Yang, Yoonna Jang, and Heuiseok Lim. Nameless Tokenization: A Lossless Tokenizer-Level Defense Against Control-Token Forgery in Open-Weight LLMs, 2026.

[46] Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. Prompt Injection as Role Confusion, 2026.

[47] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, et al. ShieldGemma: Generative AI Content Moderation Based on Gemma, 2024.

[48] Yan Zhan, Yunze Song, Mengkai Hou, Wanting Zhang, Shaobo Liu, and Zhijun Gao. Same Bytes, Different Authority: Reserved-Token Representations in Chat-Template Prompt Injection, 2026.

[49] Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, et al. Qwen3Guard Technical Report, 2025.

[50] Luman Zhao, Minghui Xu, Yue Zhang, and Yijun Yang. LTBD: Learnable Trust-Boundary Delimiters for Prompt Injection Defense, 2026.

[51] Yuanbo Zhou, Changjia Zhu, Junyu Wang, Xu He, Yan Zhai, Kun Sun, Mingkui Wei, and Junjie Xiong. Prompt Overflow: What the Guardrail Inspects Is Not What the Model Infers, 2026.

[52] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models, 2023.

A Protocol and Pre-registered Outcomes

We froze the first protocol before the confirmatory run and did not edit it after seeing results, and we record every deviation from either protocol. These runs are exploratory and fall outside both frozen protocols: the wider set of 100 requests, the three-decoy comparison, the softened-query control, the forged-query (attacker knowledge) variants, the forged-verdict and split-payload controls, the real-control-token probe, the colon-label detector and label-normalisation tests, the clean-copy ablation and the GCG control without a forged pair. The second protocol was frozen before any test-split run and has four recorded deviations.

- D1 adds the variability design: three instantiations per item and family, two paraphrased deployment policies on six families, and three training seeds per defense condition.
- D2 replaces Qwen3Guard's conversation-only native forgery with the full-task forgery found on the train split, and adds the length control.
- D3 trims compute: 5,000 training items with two attacks each, three seeds for B+T and two for T and naive adversarial training, GCG on 20 items at 250 steps, and structural search on 50 items.
- D4 was recorded after a first round of test-split defense runs and before the reported ones, so it is a deviation made with test results in view. The fixed-decoy conditions of Table 7 come from that first round. D4 draws training decoys and forged queries from one half of each pool and evaluates on the other half, adds the score-guided search and the two training-free baselines, and replaces the D3 seed plan with a smaller set to save compute: two seeds for T and one for B+T and the clean-copy target.

Table 6 lists every prediction with its outcome.

Table 6: Pre-registered predictions and outcomes. P: confirmatory study; M: benchmark; F, A: defense. Intervals are 95%.

	prediction	outcome
P1	instruction-only $\leq 10\%$	confirmed (0%)
P2	forged pair $\geq 70\%$	confirmed (100%)
P3	harmful decoy lowers bypass	not a valid test as run: the panel's "harmful decoy" scores 0.03 alone, so it is the borderline decoy; bypass fell to 29–35%; the exploratory three-decoy comparison and M3 address the prediction
P4	not recency	confirmed (append 100%, prepend 15%)
P5	syntax not load-bearing	confirmed (all variants high, no labels 0%)
P6	wide policy-dependence	refuted: median 99%, range 87–100%
P7	directive does not fix	confirmed (unchanged)
P8	cross-model transfer	confirmed (30/30, 29/29)
P9	escaping fixes it	refuted: bracket-escape 88%, label-strip 91%, nonce-fence 93%
M1	a core family $\geq 70\%$ on every guard	confirmed; best family 93.4–100%
M2	instruction-only $\leq 10\%$ on every guard	not confirmed; $\leq 7.0\%$ on four guards, 20.2% on ShieldGemma
M3	dose–response, $\rho \leq -0.8$ and non-increasing bins (Shieldstral, Qwen3Guard)	partly supported on Shieldstral: $\rho = -0.84$, interval $[-0.93, -0.67]$ over decoys ($[-0.85, -0.82]$ over requests), -0.78 on the dev split, bins not monotone; not run on Qwen3Guard
M4	control-token syntax $\geq$ angle; gap closes with special parsing off	partly supported; higher in 7 of 8 comparisons but clearly so only on Shieldstral; the parsing-off arm was not part of the benchmark run
M5	exploratory: category, source, size	released with the artifacts; size not varied within a series
F1	B alone leaves held-out bypass $\geq 50\%$	confirmed; 76.8% [75.8%, 77.8%]
F2	T generalises worse than B+T on held-out families	confirmed by the pre-registered criterion, with an effect under one point; T − B+T = +0.7 points $[-0.1, +1.5]$, one-sided $p = 0.04$ ($n = 290$)
F3	B+T held-out bypass $\leq 10\%$	confirmed; 0.9% [0.7%, 1.1%]
F4	B+T AUROC within 0.01, recall within 2 points (on the interval of the difference)	not confirmed; AUROC $-0.010\,[-0.041, +0.021]$, recall $-0.1\,[-5.0, +4.8]$: point estimates inside the margins, intervals wider
F5	B+T over-refusal within 2 points (on the interval of the difference)	not confirmed; XSTest-safe $-2.7\,[-11.1, +5.4]$, OR-Bench-hard $+0.25\,[-6.6, +7.1]$, structured benign $0.0\,[-3.0, +3.0]$: no point estimate rises by more than 0.25, intervals wider than the margin
F6	naive adversarial training costs $\geq$ 3 points structured-benign over-refusal or generalises worse	not confirmed with the fixed decoy (0.0% on a set overlapping its hard negatives, 0.9–1.1% held-out); its costs are clean FPR and the trailing-pair blind spot
A1	GCG against B+T $\leq 30\%$	not tested on B+T; on T GCG clears 20/20
A2	structural search against B+T $\leq 20\%$	confirmed; 4/50 (8% [3%, 19%]); the score-guided search, added in D4 without a prediction, clears 34/50
A3	LLM attacker	not run

B Benchmark Grammar

The attack grammar has the axes below. The full grammar yields 659 forms (654 attacks, 5 controls), and the core panel has 63. A family is held out if its syntax is JSON, YAML, markdown bold or control token, if its language is neither English nor Spanish, or if it has at least three forged pairs. Spanish-label families are therefore held out only through the syntax or pair-count rule. The defense is trained on the remaining families.

- label syntax: guard-native tags, angle brackets, square brackets, markdown, plain colon, abbreviations, JSON, XML, YAML, control tokens;
- number of forged pairs: 1–5; position: append, prepend, middle;
- decoy harm: benign, borderline, harmful; forged query: exact, paraphrase, generic, out-of-domain; language; encoding: plain text or real control tokens.

Per-family, per-category and per-source bypass and threshold sensitivity are released with the artifacts.

C Defense Implementation and Training

Bounded format. The bounded format renders the stock chat template with a placeholder document and splits it into a trusted prefix and suffix. The content is encoded on its own with special-token parsing disabled, and the two reserved boundary tokens (unused in pretraining) are inserted around it. The template puts a space before the document, which the tokenizer would otherwise merge into the content's first token. We move that space into the trusted prefix, so plain documents tokenize exactly as they do in the stock format (verified on 1,600 documents: AdvBench rows 200–299 and the clean test sample).

Training. We train a LoRA (rank 16, $\alpha = 32$) on the language model's attention and MLP projections with AdamW at learning rate 10^{-4}, batch 8, one epoch and maximum length 384. The loss is binary cross-entropy between the yes/no logit margin and the segment-max teacher target. LoRA does not touch the embeddings. In B+T a trainable two-row offset, initialised at zero, is added to the embeddings of the two boundary tokens (ids 36 and 37), and every other embedding stays frozen. The data are 5,000 train-split prompt items, each clean and with two training-family attacks, plus 1,250 structured-benign items, for 16,250 examples. One epoch is 2,032 steps, the last batch partial. Each attacked example's decoy and forged query are a seeded draw from the training halves of the pools: 20 of the 40 benign decoys, 3 of 6 borderline and 4 of 8 harmful decoys, and 4 of the 8 forged queries at each relatedness level. The hard negatives are train-split structured-benign items. Naive adversarial training and the clean-copy ablation use the same examples with different targets.

Compute and adaptive budgets. Training and the test-split defense, adaptive and mechanism runs used one NVIDIA RTX 4090. A training run takes about 17 minutes (1,028 s for T and 1,037 s for B+T at the last logged step, 2,030 of 2,032). The benchmark ran on an Apple-silicon laptop GPU. The score-guided search has 300 guard calls per item on 50 items, starting from 20 random candidates and then mutating batches of 10 children with restart probability 0.2. The structural search scores all 684 attacks on each of 50 items (batch 128). GCG optimises a 20-token suffix inside the forged document for 250 steps (batch 32 candidates per step) on 20 items, and stops on an item at its first success.

Fixed-decoy conditions. Table 7 gives the conditions trained and tested with one fixed decoy and query per family: naive adversarial training and the generality runs of T on Llama Guard 3 1B and Qwen3Guard 0.6B (Section 6). They have no held-out-decoy column. The hard negatives of the trained rows include 380 of the 500 structured-benign test items, so their structured-benign column is not a held-out measure.

Table 7: Fixed-decoy conditions on the test split, $\tau = 0.5$, in %. Columns as in Tables 3 and 4; naive adversarial training has two seeds (range), the other rows seed 0.

	bypass ↓		detection			over-refusal ↓			benign + forged pair	
condition	train	held-out	AUROC	recall	FPR	OR-B.	XST.	struct.	benign	harmful
naive adv. training (2 seeds)	0.9–1.0	0.9–1.1	0.921–0.925	83.3–84.6	12.6–14.9	0.3–0.8	8.0–10.7	0.0	7.8–8.2	8.0–8.6
Llama Guard 3 1B, base	84.0	82.0	0.758	58.7	16.9	40.5	1.3	23.4	2.4	97.3
Llama Guard 3 1B, T	2.3	2.5	0.771	58.4	15.0	38.3	1.3	23.0	22.5	100
Qwen3Guard 0.6B, base	28.3	9.5	0.917	88.5	23.6	76.8	21.3	30.4	10.5	100
Qwen3Guard 0.6B, T	0.0	0.0	0.924	89.0	23.3	77.3	20.0	29.4	25.9	100

OR-B.: OR-Bench-hard; XST.: XSTest-safe; struct.: structured benign; benign / harmful: decoy of the forged pair on benign content.

Content tokens. Disabling special-token parsing leaves atomic any added token that the tokenizer does not mark special [45, 48], so we checked which added-vocabulary ids the content encoder can still emit. For each guard tokenizer we encoded every added token's string on its own, embedded in plain text and in 500 random concatenations, with special-token parsing disabled, and recorded any added or special id in the output. No added or special id is reachable from content on Shieldstral, the guard the defense is trained on (1000 added tokens, all special, with the boundary tokens at ids 36 and 37), or on Llama Guard 3 1B (256, all special). On Qwen3Guard-0.6B, 12 of 26 added tokens are not marked special and stay atomic, among them the tool-call tags <tool_call> and </tool_call> and the fill-in-the-middle markers. On ShieldGemma 2B, 242 of 249 are not special (the <unused*k*> and other reserved markers) and also stay atomic. Content can produce these ids, so a boundary-token design on these tokenizers must not use them as boundary tokens.

D Reproducibility

The runs used these Hugging Face revisions: Shieldstral-1.0-3B `003ec7e`, Llama Guard 3 1B `acf7aaf`, Qwen3Guard-Gen-0.6B `fada3b2` and ShieldGemma 2B `d1dffc9`. Local guards are scored from a single forward pass with no sampling. The score is the probability of the unsafe label renormalised over the guard's label tokens, computed in fp32 from bf16 hidden states. We reached the API-served guards, gpt-oss-safeguard-20B and Llama Guard 4 12B, through OpenRouter at temperature 0 without pinning a provider. Every benchmark call to gpt-oss-safeguard was served by Groq, as recorded in the response metadata; the cross-model runs did not record the provider. Dataset sources and licenses: AdvBench, HarmBench, Aegis 2.0, BeaverTails, the OpenAI moderation evaluation and ToxicChat, with XSTest and OR-Bench as over-refusal sources. We use the non-commercial sets (ToxicChat, BeaverTails) for research only. All hyperparameters, splits and the full protocol are in Sections A and C.

E Open Science

We release the GuardRebind benchmark and harness, the attack grammar, the defense code, the training commands and data recipe, and the result files behind every number in this paper, at https://github.com/ukanwat/guard-policy-injection. The released results contain item ids and scores, plus text only from the public benchmarks we used (such as the AdvBench requests of the confirmatory run and the benign development-split prompts that won as search decoys) and model outputs that quote it. We release no new harmful content. The pre-registered protocol and its deviations are included so that predictions can be checked against outcomes.